PART 3 REWRITE REPORT DRIVE LAUNCHER

Drive Launcher Rewrite

What changed on main, and why: the ownership problem you hit, researched and solved; all 41 findings from Part 1 fixed; the design from Part 2 built and tested; and the new problems the rewrite turned up, with what was done about each.

DRAWING	STATUS	BUILDS ON
Changes, research and verification	Committed on main, not pushed	Part 1 (findings F1–F41) · Part 2 (design)
DATE	**CHECKS**	**LEVEL**
16 October 2026	466 tests pass · typecheck, lint and build clean	Same as Part 2: terms are explained where they first appear

41/41 FINDINGS FIXED · **466** AUTOMATED TESTS · **16** COMMITS ON MAIN · **27/34** NEW PROBLEMS FIXED OR MITIGATED · **2** DECISIONS FOR YOU

01 The short version

- **Your diagnosis of the lock problem was right.** On Google Drive only a file's owner can delete it; everyone else can only change it. Your idea of emptying such files and letting the owner's app delete them is now how the app deletes. It needed no shared list: each emptied file carries a hidden tag, and the owner's app finds its own tagged files with one search (section 02).
- **All 41 findings from Part 1 are fixed**, each one listed with where it was fixed and how it was checked (section 07).
- **The app now has the structure from Part 2.** One controller owns the hosting session, the lock is held and renewed for the whole session, the sync compares three versions of every file (last sync, this PC, the cloud) and asks before overwriting anything, and an upload only counts once its manifest is written, last.
- **Transfers run in parallel lanes** that adapt to your connection and to Google's limits, update files in place instead of re-creating them, and survive a crash: the next start finishes the upload without creating duplicates.
- **Everything that can be tested without your Google account is tested**: 466 automated tests, including two simulated PCs taking turns on an in-memory Google Drive that enforces the same ownership rules as the real one. What still needs a real account and a real Minecraft server is listed as a checklist in section 09.
- **16 commits on main**, starting with your own branch, which was saved and merged first. Nothing was pushed (section 10).

THREE THINGS ONLY YOU CAN DO

1. In Google Cloud Console, switch the OAuth consent screen from *Testing* to *In production*. In Testing mode every login expires after 7 days (section 02).
2. Run the two-account checklist in section 09 before you give the build to your friends.
3. Decide on the two open design questions in section 08: what to do when a member's unfinished upload blocks the server for good (N21), and whether the app may keep accepting the Minecraft EULA for you (N25).

02 The ownership problem you ran into

You described it like this: you could not delete files that other members had uploaded, but you could change their contents. Your plan was to overwrite such files with nothing, keep a list of them, and let each file's owner delete them automatically the next time they open the app. I checked this against Google's current documentation (pages updated September 2026), and had every claim checked a second time.

What Google's documentation says

QUESTION	ANSWER	SOURCE
Can a member delete or trash a file someone else uploaded?	**No.** "Only the file owner can trash a file." Anyone else, even with the Editor role, gets `403 insufficientFilePermissions`. This includes the person who created the shared folder.	developers.google.co
Can they change its content?	**Yes.** "If you upload a new version of a file owned by someone else, the original owner remains the same."	support.google.com/d
Can they rename it, tag it, move it?	**Yes.** The roles table allows writers to modify metadata and to move items inside the shared folder (moving needs share rights, which editors have by default).	…/api/guides/ref-rol
Who pays for the storage?	The file's **owner**, always. A file you upload into someone's folder counts against *your* 15 GB; new content you write into *their* file counts against theirs.	support.google.com/d
Does a 0-byte overwrite free the space?	**Practically yes.** A file's quota usage is its current version plus versions pinned "keep forever". Older versions are purged within about 30 days. The app never pins versions.	…/reference/rest/v3/
Does "Remove" in the Drive website delete it?	**No.** The file moves to its owner's My Drive and keeps using their storage. Other members lose sight of it.	support.google.com/d

So your diagnosis was exactly right, and so was the core of your idea: a non-owner can empty the file, and only the owner can finish the job. The research added a few improvements, which the rewrite uses.

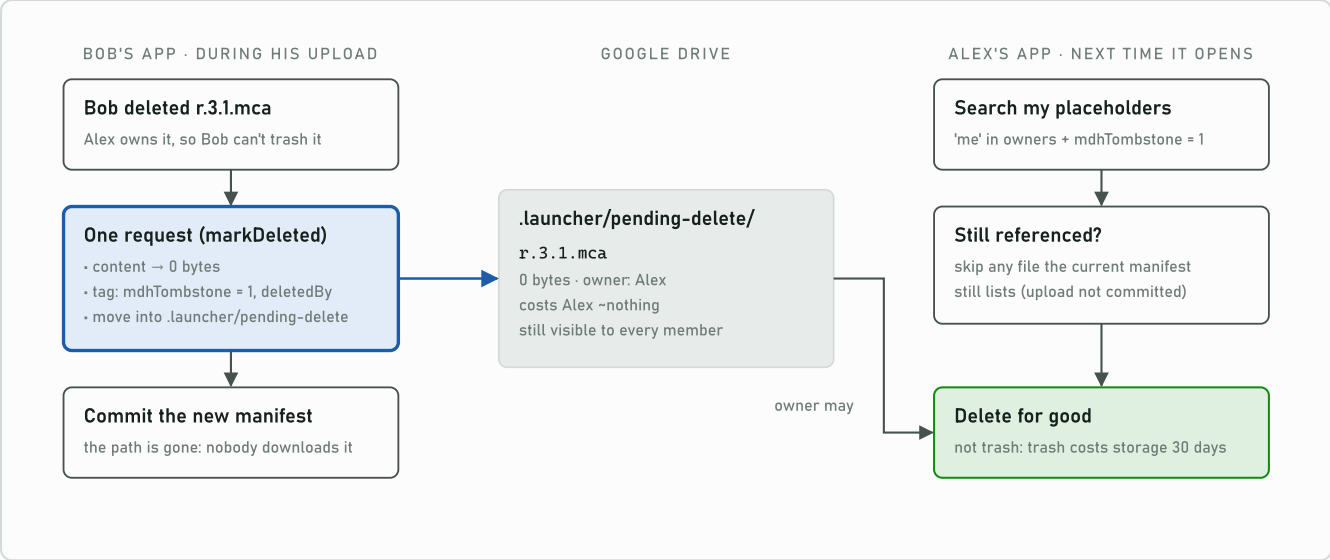


Figure 1. Your idea with the research's refinements. Nothing shared has to be written to track placeholders, because Drive itself can find them by their tag. Only Alex's own app ever deletes Alex's files.

What changed compared with your plan

- **No shared list.** A list file that several members edit would have its own race conditions and its own owner. Instead each placeholder carries an app-private tag (`appProperties`, visible only to this app), and the owner's app finds its own placeholders with one search.
- **Out of the way, not out of reach.** Placeholders are moved into `.launcher/pending-delete/`, so the world folder stays clean. Unlike the website's "Remove", the file stays inside the shared folder, so every member can still see it and the app can say "3 deleted files are waiting for Alex to open Drive Launcher" (you can see that line in Figure 4).
- **Delete permanently, not to the trash.** Trashed files keep counting against the owner's storage for 30 days.
- **Never delete something a manifest still uses.** A placeholder is created during an upload, before that upload's manifest is committed. If the owner's app ran its cleanup in that window and the upload then failed, the old manifest would point at a deleted file. The cleanup therefore skips any file the current manifest still lists.
- **Deletions are rare.** Every change is written into the existing file (same Drive ID, same owner), so only files that really disappear from the world need any of this.
- **Deleting a whole server.** Before trashing the folder, the owner's app tags every file it does not own, so each member's app later deletes their own leftovers instead of Drive silently moving them into their My Drive.

Here is the decision the transfer engine makes for every deleted file. It is the whole rule in one place:

ELECTRON/TRANSFER/TRANSFERENGINE.TS · DELETEREMOTE() (SHORTENED)

```
const meta = await drive.getMeta(job.driveId);           // 404 → already gone, fine
if (meta.capabilities.canTrash) {                        // I own it
    await drive.trash(job.driveId);                      // 30 days to undo a mistake
    return outcome('trashed');
}
if (meta.capabilities.canModifyContent === false) throw new AppError('FORBIDDEN', …);
await drive.markDeleted(job.driveId, {                   // someone else owns it
    mdhTombstone: '1', mdhServer: job.serverId,
    mdhDeletedBy: me.email, mdhDeletedAt: now,
}, { addParent: pendingDeleteFolder, removeParent: meta.parents[0] });
return outcome('placeholder');                           // 0 bytes, waits for its owner
```

The old app never deleted anything on Drive. A file you deleted locally stayed in the cloud and came back with the next download.

Alternatives that were checked and rejected

OPTION	WHY NOT
Transfer ownership to the host	Possible between personal accounts, but every file needs the recipient's consent and sends an email that cannot be turned off. All storage lands on one person, and the other members still cannot delete afterwards.
Shared drive	Solves it completely (files belong to the drive, content managers can trash anything), but personal @gmail.com accounts cannot create one. Worth revisiting only if one friend pays for Google Workspace Business Starter.
"Remove from folder"	The file moves to the owner's My Drive and keeps costing them storage; other members can no longer see or verify it.
The drive.file scope	Would avoid Google's "restricted scope" rules, but each member's app could only see files that member created or picked by hand. Syncing a world needs the full drive scope.

Things the research found that you should act on

- **Publish the OAuth app.** In Google Cloud Console, set the OAuth consent screen's publishing status to "In production". In "Testing" mode every login expires after 7 days. A small group of friends falls under Google's personal-use exception, so verification is not needed; members see a one-time "unverified app" warning, and the project is limited to 100 users over its lifetime. The app now recognises the expiry (`invalid_grant`) and sends you back to the login page with a clear message instead of failing on every request.
- **A full Drive blocks everyone.** When a file's owner is over quota, nobody can change that owner's files. The app now reports this with the owner's name (`QUOTA_FULL`) instead of failing silently.
- **Storage falls on whoever uploaded first.** Most world files are created by the first host, so that person's 15 GB pays for most of the world from then on. Worth knowing when choosing who seeds a new server.

03 What the code looks like now

The layers are the ones from Part 2, Figure 1, now with the real module names. Arrows only point down: a layer uses the ones below it and never the ones above. Information that travels up ("the upload is 40% done") travels as events. A script over all 82 modules found **no import cycles** (Part 1's F29 was two services importing each other).

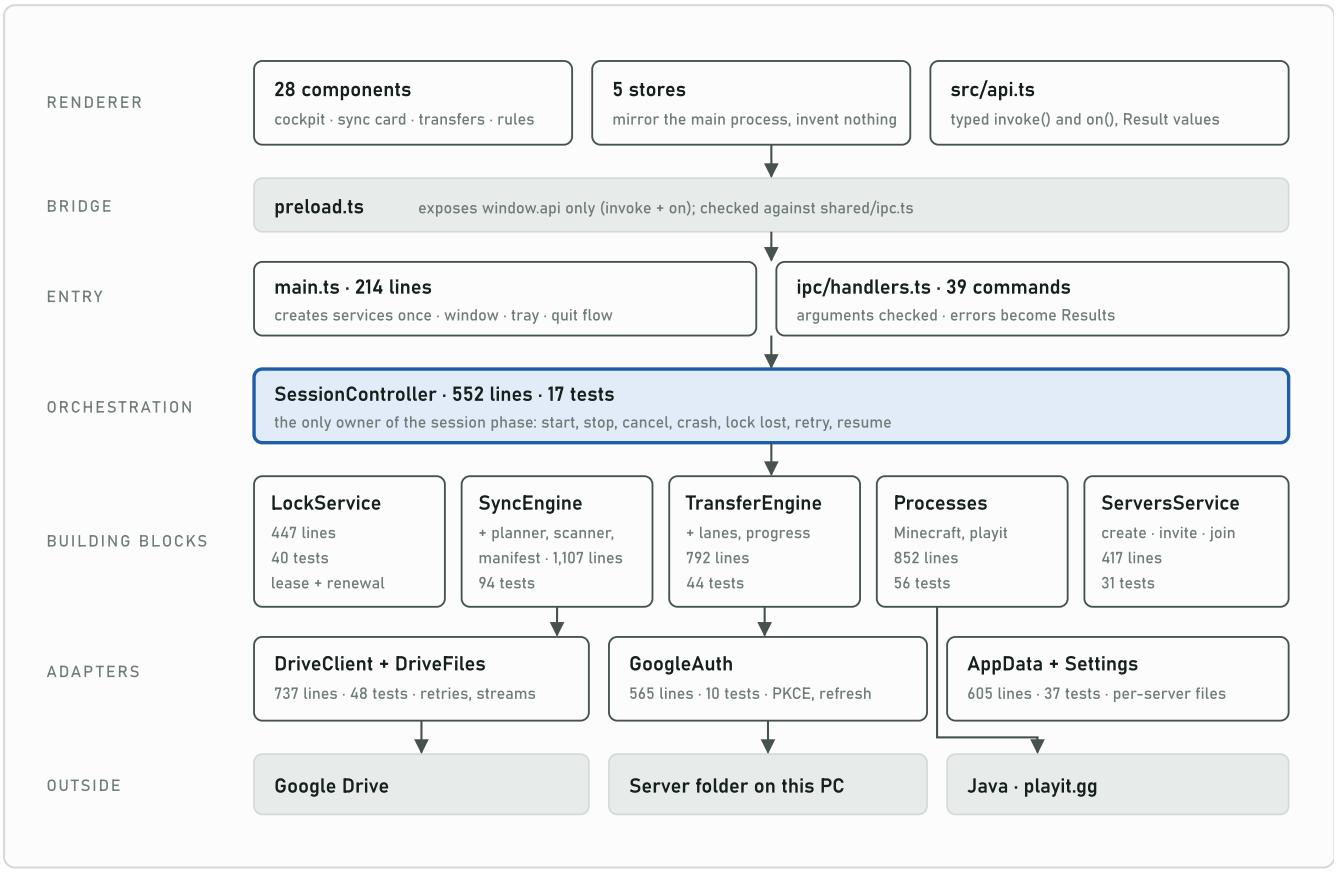


Figure 2. The rewritten app. Every box in the bottom three layers gets its dependencies through its constructor, so tests hand it fakes: an in-memory Google Drive, a fake Java process, a temporary folder. The old `electron/services/` folder is gone.

AREA	BEFORE	AFTER
Session	Flags spread over main.ts and the stores; Stop ran twice	One state machine; every action allowed only in certain phases
Lock	Read wrote it; 60 s lease ran out mid-transfer; PC clocks	Read never writes; renewed every 15 s for the whole session; Google's clock
Change detection	Version counters; changed files were never downloaded	MD5 of every file; three-way plan (last sync, this PC, cloud)
Manifest	Never reached Drive; file names without folders	`.launcher/manifest.json`, full paths and Drive IDs, written last as the commit
Overwrites	Download replaced local edits silently	"Both changed" card with keep-cloud (backups first), keep-mine, combine
Transfers	One file at a time, file read into memory, extra requests	Adaptive parallel lanes, streamed, in-place updates, resume without duplicates
Deletes on Drive	Never applied; non-owners could not delete	Owner trashes, others leave a 0-byte placeholder the owner deletes
Ignore rules	Private per PC, separators broke nested paths	Shared `.launcher/sync-rules.json` plus built-in rules in code
IPC	41 string channels, three error styles	One typed contract, every answer a `Result`, arguments checked
Build	Did not compile	`tsc`, ESLint and `vite build` clean; `npm test` runs 466 tests

04 One session, start to finish

A **state machine** is a fixed list of phases plus the rules for moving between them. The session controller is one: the Start button only works in *idle*, Stop only in *running* (or, while starting, it cancels). That alone makes Part 1's "Stop runs twice" impossible. Each phase the lock should show is written into `lock.json`, so other members see "Steve is uploading the world" instead of a guess.

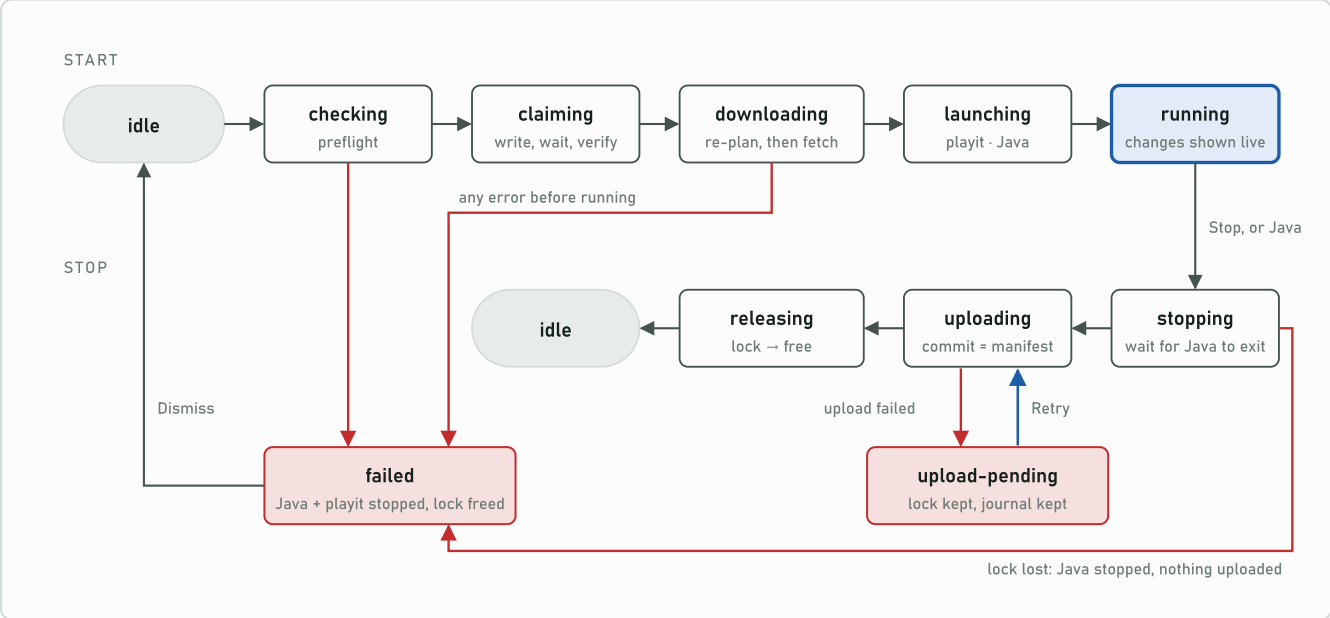


Figure 3. The phases of a session. Red arrows are the failure paths; each of them has a test in `SessionController.test.ts`. Stop pressed while starting cancels the start and returns to idle.

What an upload does, and what happens if the app dies at each step

The upload is written so that the app can be killed at any line and the next start still ends in a correct state. The trick is the order: the cloud only "has" a new version once `manifest.json` says so, and that file is written last. A small **journal** file on this PC remembers that an upload was started.

#	STEP	IF THE APP DIES RIGHT AFTER THIS STEP
1	Scan the folder and plan against the last sync (base)	Nothing was written. The next start plans again.
2	Write the journal (this PC, server, base revision, session)	The next start sees the journal and finishes the upload before anything else.
3	Create missing folders, parents first	The resumed upload finds them by name and reuses them.
4	Transfer: update changed files in place, create new ones (tagged with server and path), delete or leave placeholders	The resumed upload finds files it already created by their tag and files already updated by their MD5, so nothing is uploaded twice. Other members see "unfinished upload" and wait.
5	Read the cloud manifest again; stop if someone else committed meanwhile	Same as 4.
6	**Write the new manifest (the commit)**	The resumed upload sees a manifest written by this account on top of the journal's revision, recognises its own commit and only writes the base.
7	Write the base, delete the journal, release the lock	Done. If the release did not happen, the lock simply expires after 45 s.

Steps 4 and 6 have tests that simulate exactly these crashes: *"a failed upload commits nothing, keeps the journal, and the retry creates no duplicates"* and *"an upload whose commit landed before the app stopped is finished, not reported as a conflict"*.

05 Before and after

Four changes, side by side, that carry most of the difference. The old code is from your merged branch (commit `33bb9d1`), shortened only where marked.

A Map that was read as an object (F1)

BEFORE

ELECTRON/SERVICES/FILETRANSFERSERVICE.TS (33BB9D1)

```
function buildTransferMap(filesToBeTransfered: Map<string, IFileEntry>) {
    const root: TFolderNode = { type: "folder", children: new Map() }
    for (const [filePath, fileEntry] of Object.entries(filesToBeTransfered)) {
        // never runs: Object.entries() of a Map is []
        …
```

AFTER

ELECTRON/SYNC/SYNCENGINE.TS · DOWNLOAD() (SHORTENED)

```
const jobs: TransferJob[] = [
    ...download.map((p) => ({ kind: 'download', path: p, driveId: remote.files[p].driveId,
                              size: …, md5: …, dest: this.abs(root, p) })),
    ...deleteLocal.map((p) => ({ kind: 'delete-local', path: p, target: this.abs(root, p) })),
];
const result = await this.d.transfers.run('Downloading', jobs, signal);
if (result.failed.length > 0) throw new AppError('DOWNLOAD_INCOMPLETE', …);
```

The plan is a flat list of jobs built from plain arrays, so there is no tree to build and no Map to misread. A transfer that does not move every file can no longer report success.

Uploading while Minecraft is still saving (F5, F8)

BEFORE

ELECTRON/SERVICES/CHILDRENPROCESSES.TS + SERVERSERVICE.TS (33BB9D1)

```
export function killServer() {
    serverProcess?.stdin?.write("stop\n");     // asks Java to stop, does not wait
}
…
await updateLockFile(folderId, "stopping")
if (shouldUpload) {                            // a setting, not the session's state
    const uploadRes = await uploadServerFolder(folderId)   // Java is still writing
```

AFTER

ELECTRON/SESSION/SESSIONCONTROLLER.TS · RUNSTOP() (SHORTENED)

```
await this.enter('stopping');
await this.d.changes.stop();
try {
    await this.d.minecraft.stopAndWait(60_000);   // "stop", wait for exit, kill after 60 s
} catch (err) {
    // Java could not be stopped: uploading now could save a half-written region file
    this.setState({ phase: 'upload-pending', error: toAppError(err) });
    return;
}
await this.d.playit.stop();
if (this.lockLost) return;                        // someone else holds the server now
await this.upload(serverId);                      uploading → releasing → idle
```

Reading the lock wrote it (F11, F34)

BEFORE

ELECTRON/SERVICES/SERVERSERVICE.TS · GETSERVERLOCK() (33BB9D1)

```
if (!data.files || data.files.length === 0) {
    lockInitializing = true;
    const res = await updateLockFile(folderId, "offline")   // a read that writes
    …
// elsewhere: expiry compared with this PC's clock (Date.now())
```

AFTER

ELECTRON/LOCK/LOCKSERVICE.TS · INSPECT() (SHORTENED)

```
const file = await this.readConsistent(serverId);       // metadata + content, no writes
if (!file) return { state: 'free', lock: null };
const lock = parseLock(file.text);
// Age by Google's clock: the file's modifiedTime against the Date header of the same answer
const ageSeconds = Math.max(0, (file.serverNow - Date.parse(file.modifiedTime)) / 1000);
if (claim && lock.sessionId === claim.sessionId) return { state: 'mine', lock, ageSeconds };
return { state: ageSeconds > lock.leaseSeconds ? 'stale' : 'held', lock, ageSeconds };
```

The lock file is created only by a claim, under the session. The reason the old read wrote, as you explained, was that a stale lock left by someone else could not be deleted. A lock is never deleted now: it is set to `free` in place, which any member may do, and a new claim overwrites it in place.

One typed contract instead of string channels (F26, F31)

BEFORE

RENDERER (33BB9D1)

```
const result = await window.ipcRenderer.invoke("drive-invite-user", serverId, email, message)
if (!result.success) { alert(result.error); return }     // the chosen role was never sent
```

AFTER

SHARED/IPC.TS + RENDERER

```
'servers:invite': (serverId: ServerId, email: string,
                   role: 'writer' | 'reader', message: string) => void;
…
const res = await request('servers:invite', serverId, email.trim(), role, message);
// a wrong channel name, argument or result type is a compile error;
// errors arrive as { ok: false, error } and are shown as a notice
```

06 What you see in the app

The layout and look are yours. What changed is what the screen knows: a card under the server picker says what Start would do, a transfer shows its lanes, and choices appear only when they are needed. These are screenshots of the real built renderer at the window size, fed with sample data (no Google account was used).

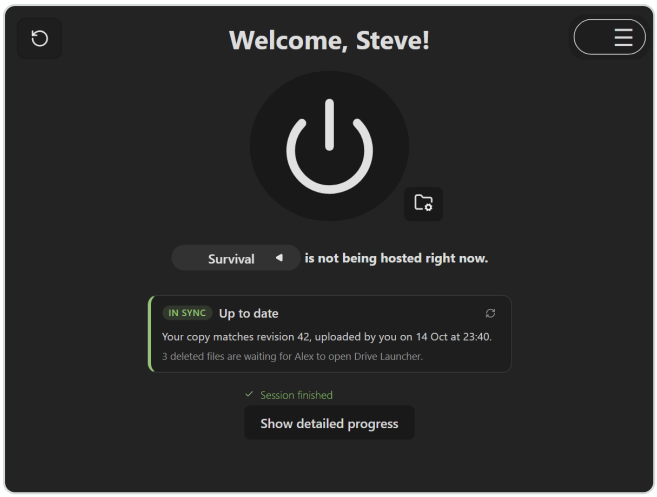


Figure 4. Idle and in sync. The last line is the placeholder count from section 02.

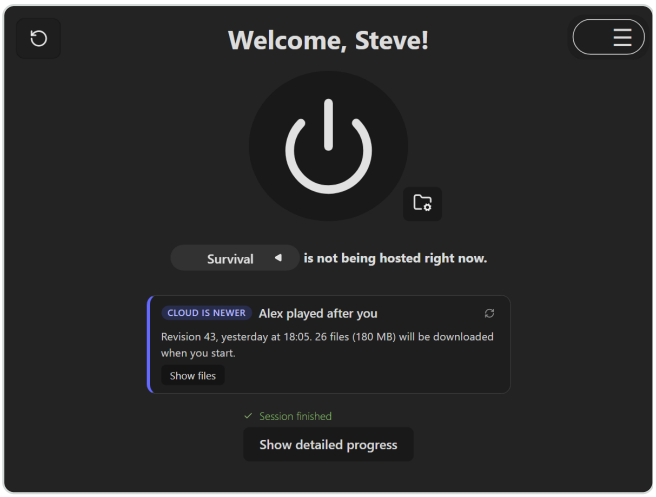


Figure 5. Someone played after you: what will be downloaded, and how much.

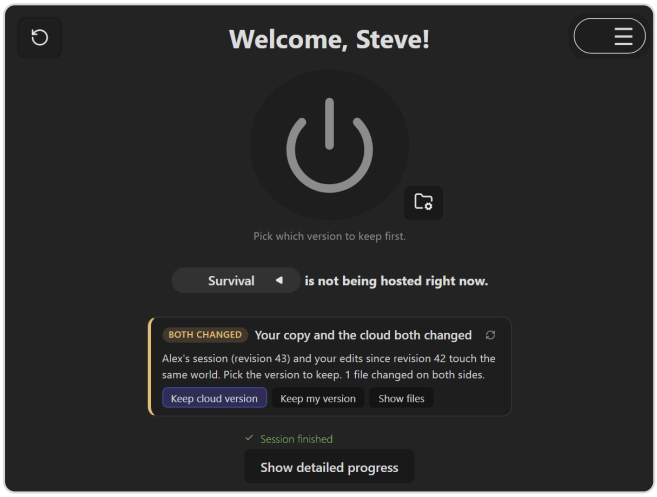


Figure 6. Both sides changed the same file: Start waits for a choice, and Combine is not offered.

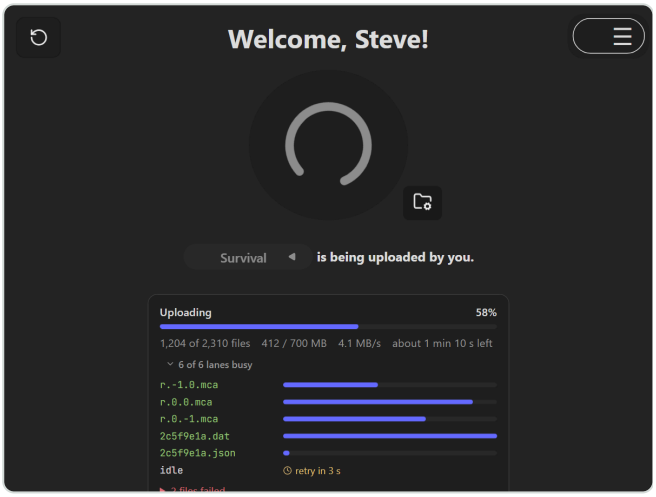


Figure 7. An upload: overall progress, speed, time left and each lane's file.

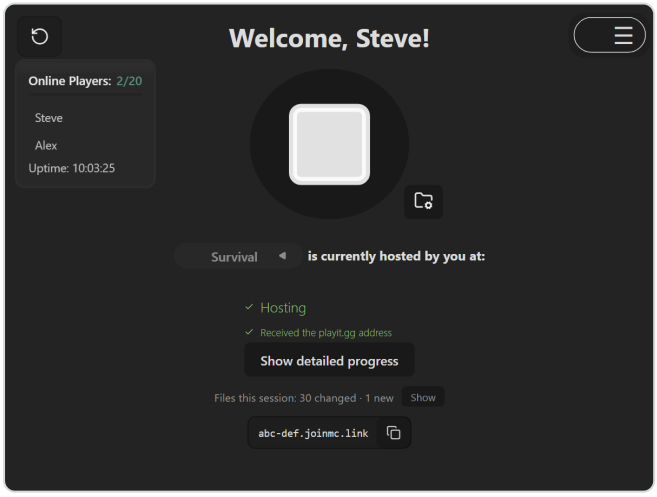


Figure 8. Hosting: players, uptime, this session's changes and the address to share.

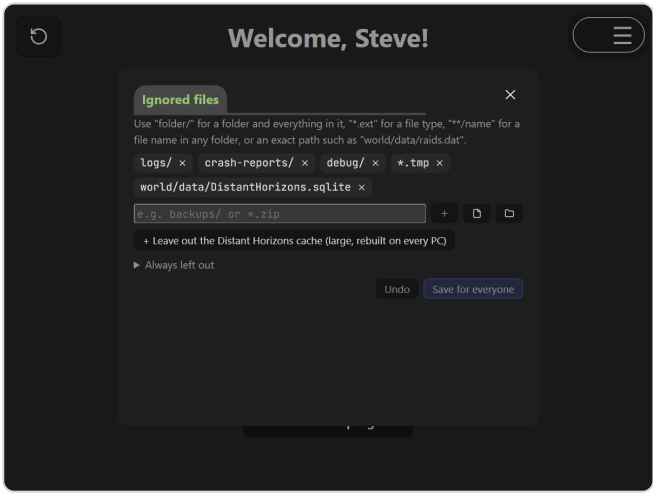


Figure 9. The shared ignore rules: one list for every member, stored on Drive.

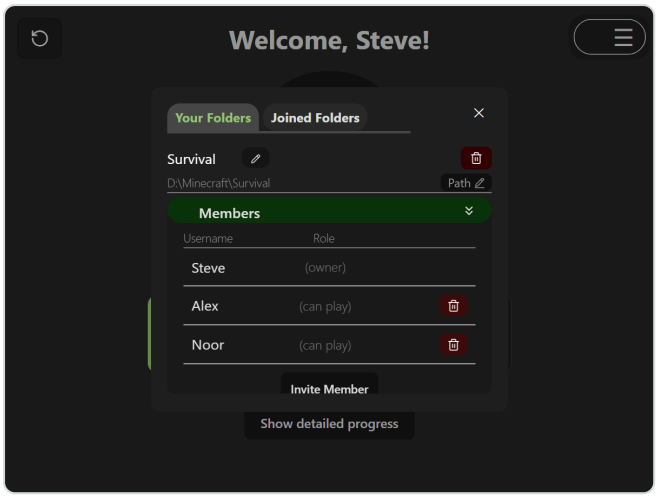


Figure 10. Drive Folders. The manual Download and Upload buttons are gone (F10).

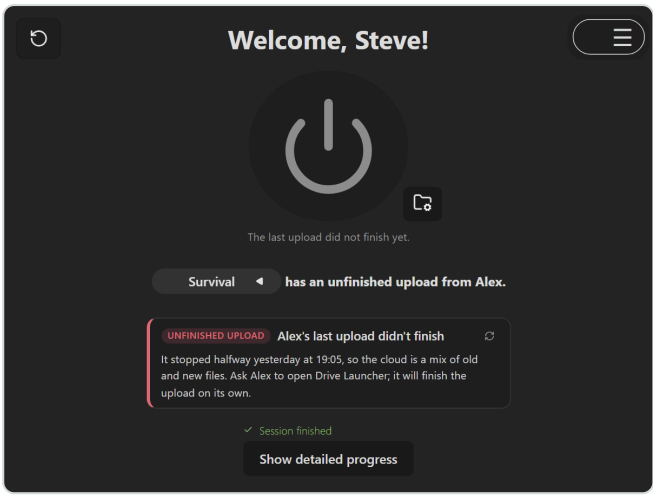


Figure 11. Another member's upload stopped halfway: the server waits for their app (see N21).

07 Every finding from Part 1

All 41 are fixed. "Checked by" names the automated test that proves it where one exists; the rest were checked by reading the code or by the build itself.

ID	FINDING	FIXED IN	CHECKED BY
F1	The transfer list is always empty	SyncEngine builds flat job lists	SyncEngine: new server uploads everything
F2	manifest.json never reaches Drive	.launcher/manifest.json, written last	SyncEngine: revision 1 readable by the other PC
F3	Changed files are never downloaded	MD5 three-way planner	planner tests; SyncEngine: changes reach the other PC
F4	Manifest entries use the bare file name	Full POSIX paths + Drive IDs	manifest tests; SyncEngine: files land in the right folders
F5	The upload starts while Minecraft is still saving	stopAndWait, then upload	MinecraftProcess: sends stop and waits; SessionController
F6	The lock's lease runs out during transfers	Renewal every 15 s for the whole session	LockService: renewal tests
F7	Start only refuses "online"	Claim refuses any held lock; claim-then-verify	LockService: claim tests
F8	A lapsed lock makes the host skip the upload	Upload decided by the session phase	SessionController: crash and lock-lost paths
F9	The stop sequence runs twice	stop() only acts in running; one promise	SessionController: stop is idempotent
F10	Manual Download and Upload ignore the lock	Removed; only sessions transfer	UI (Figure 10)
F11	Reading the lock writes it	inspect() never writes	LockService: never writes; SyncEngine: preflight never writes
F12	Async steps that nobody awaits	Everything awaited or handed to a logger	Code review; tests fail on unhandled rejections
F13	The upload ignores its own safety check	Refuses when the cloud moved on; deletes applied	SyncEngine: refuses to overwrite a newer cloud
F14	One in-memory manifest for every server	Per-server base and journal files	appData tests
F15	Downloads overwrite local edits without warning	"Both changed" + choices + backups	SyncEngine: keep-cloud backs up my version
F16	Downloaded files will look modified	Base stores local modification times	SyncEngine: second PC ends in sync
F17	Starting the watcher floods the app	No initial scan events; batches; display only	changeTracker tests
F18	Big files read and written synchronously	Streams everywhere	scanner, DriveFiles tests
F19	Transfers are sequential and use extra requests	Adaptive lanes; one request per small file	TransferEngine, LaneController tests
F20	The project doesn't build	Types fixed; typecheck script	`tsc` and `vite build` pass
F21	Waiting for playit.gg can hang or escape	Timeouts; cleanup in finally	PlayitProcess: timeout kills the agent
F22	Built-in files aren't ignored on a fresh install	Built-in rules in code	paths tests
F23	A nested file picked with Browse never matches	Picker returns a relative POSIX rule	handlers: ruleFromPick
F24	The blacklist is global and private	Shared sync-rules.json per server	SyncEngine: rules test

ID	FINDING	FIXED IN	CHECKED BY
F25	Manifest paths use the OS separator	toPosix() for every stored path	paths tests
F26	Errors are reported three different ways	AppError → Result at the IPC edge	handlers tests
F27	The heartbeat can overlap and fail silently	setTimeout chain; failures handled	LockService: keeps renewing through errors
F28	A new OAuth client for every call	One client; single-flight refresh	GoogleAuth: refreshes once
F29	Two services import each other	Layers; dependencies injected	Cycle check over 82 modules: 0
F30	main.ts does everything	Composition root only (214 lines)	Code review
F31	IPC channels are plain strings	shared/ipc.ts contract; window.api	handlers: every command registered
F32	The same fact is stored in several places	One owner per fact; UI mirrors	Renderer lib tests
F33	The cockpit's ownership check is always true	Role from Drive (myRole)	UI; readers cannot start
F34	Lock expiry depends on every PC's clock	Google's Date header	LockService: inspect tests
F35	One vanished file fails the whole step	ENOENT tolerated where safe	scanner tests
F36	Typos that became part of the API	Renamed; token revoke fixed	GoogleAuth: logout revokes
F37	Effects that leak	One subscription point; effects clean up	Code review
F38	A failed start leaves the progress panel open	Panels follow the session state	UI
F39	eula.txt is rewritten on every launch	Written only when missing or false	MinecraftProcess: does not rewrite eula
F40	Leftovers and repeated work	Old modals, template config removed; members fetched once	ServersService tests
F41	The playit.gg check can block a start	Condition fixed	SessionController: no playit when off

08 New problems found during the rewrite

Writing each module against the real rules of Drive, Windows and Minecraft turned up problems that Part 1 did not list, mostly because the old code never got far enough to hit them. FIXED means code and a test; MITIGATED means the app now handles it clearly but the root cause is outside the app; OPEN needs a decision from you; DOCUMENTED is a fact to know.

ID	PROBLEM	WHAT WAS DONE	STATUS
N1	A manifest path like `../../Windows/x` would write outside the server folder	Manifests with unsafe paths are rejected	FIXED
N2	A file locked by another program (EBUSY) would look deleted and be deleted in the cloud	Unreadable files stop the scan with their name; transfers retry them	FIXED
N3	Windows junctions inside the server folder could loop or leave it	Skipped by the scanner	FIXED
N4	Names from other systems that Windows cannot store (`aux.dat` , `a:b` , trailing dots, names differing only in case)	The download refuses before touching anything and names the file	FIXED
N5	Server IDs are used in file paths on this PC	Validated everywhere, including every IPC argument	FIXED
N6	Two servers pointed at the same or nested local folders would overwrite each other	Refused when choosing the folder	FIXED
N7	Joining a file, your whole My Drive or the folder that holds your servers by pasting the wrong link	Refused with an explanation; your own server's link is recognised as owned	FIXED
N8	Drive allows two files with the same name; two PCs creating `.launcher` at once would split the server	Oldest copy wins; the extra one is trashed by its creator	FIXED
N9	After a crash, restarting within 45 s finds this PC's own lock "held"	The session waits out the lease and claims	FIXED
N10	A crash between the commit and the base write looked like a conflict	The journal recognises its own commit	FIXED
N11	A resumed upload re-created files the first attempt had created	Found again by their server and path tags	FIXED
N12	A file trashed by hand on the Drive website still accepts new content, but nobody else can download it	Detected by listing each folder once; uploaded as a new file	FIXED
N13	A Java process that cannot be killed would be uploaded under	No upload until it is gone; Retry refuses while it runs	FIXED
N14	Losing the lock during an upload could still commit	The commit is skipped once the session is aborted	FIXED
N15	Chat messages could fake the player list (the app parses Java's answer to "list")	Only real server replies are parsed; quoted chat is ignored	FIXED
N16	Typing `stop` in the console left the app thinking the server still ran	The exit takes the normal stop path and uploads	FIXED
N17	Writing to Java after it exited crashed the process pipe (EPIPE)	Commands refused once stdin is closed	FIXED
N18	A fresh playit.gg agent waits forever for its claim link	The link is detected and shown in the error	FIXED
N19	A run.bat from another PC can point at a Java path that does not exist here	Falls back to `java` on PATH; the app's RAM setting replaces the script's `-Xmx`	FIXED
N20	Two copies of the app on one PC would share the same files and fight over the lock	Single-instance lock; the second copy focuses the first	FIXED
N21	Another member's unfinished upload blocks the server until their app returns. Because files are updated in place, the cloud then holds a mix of old and new content	Blocked on purpose, with a clear message (Figure 11). Proposed fix: an owner-only "Repair from this PC" that lists each folder once, compares Drive's MD5 with the manifest, and re-uploads the differing files from this PC under a normal lock	OPEN

ID	PROBLEM	WHAT WAS DONE	STATUS
N22	A member who leaves keeps paying for the world files they uploaded while the server exists	The Leave dialog now says so; deleting the server hands the files back as placeholders	DOCUMENTED
N23	When the owner of `manifest.json` or `lock.json` runs out of storage, nobody can commit or claim	Reported with the owner's name (QUOTA_FULL); the owner has to free space	MITIGATED
N24	OAuth apps in Testing mode expire every login after 7 days	Expiry recognised; publish the app (section 01)	MITIGATED
N25	The app accepts the Minecraft EULA for the user by writing `eula=true`	Unchanged from before. Suggested: ask once, with a link to the EULA	OPEN
N26	Preflight and the following download hashed the whole world twice	The last scan is reused as a hint	FIXED
N27	The console pulled the whole page down on every new log line (as in the old app)	Only the console box scrolls, and stops following when you scroll up	FIXED
N28	A second, untouched electron-builder template config (YourAppID, no `config/`) next to the real one	Removed	FIXED
N29	Members invited as readers could press Start and fail at the lock	Start is disabled with the reason	FIXED
N30	Google meters Drive in "quota units" since May 2026 (per-user limits per minute)	Lanes shrink on 429 and grow back; retries use Google's backoff	FIXED
N31	Rules saved while someone else is hosting apply at that host's upload	Expected; the editor says rules apply from each app's next sync	DOCUMENTED
N32	Bottleneck's optional Redis code uses eval, so the build warns	The Redis code is never loaded; the warning is harmless	DOCUMENTED
N33	Storage falls on whoever uploaded a file first	Explained in section 02	DOCUMENTED
N34	The console font loads from Google Fonts	Falls back to a local monospace font offline	DOCUMENTED

09 What was tested, and what still needs you

Automated

Run them with `npm test` (about 6 seconds). The sync tests are the important ones: they run two PCs, each with its own folder and app data, against one in-memory Google Drive that follows My Drive's rules (only the owner may trash, anyone may change content, the uploader owns new files).

AREA	TESTS	WHAT THEY COVER
Sync	94	Planner cases, scanner fast path and errors, manifest validation, two-PC sessions, conflicts and choices, placeholders, crash recovery, Windows names
Drive + auth	58	Retries and backoff, resumable uploads, MD5 checks, placeholders, token storage, refresh, revoke
Processes	56	Launch arguments, readiness, stop and kill, player list, playit address and timeouts
Transfers	44	Lanes, throttling, retries, progress and time left, failures
Lock	40	Inspect, claim races, renewal, takeover, release, Google's clock
Servers, settings, data	68	Create, join, invite, leave, delete, folder rules, atomic files, migration of your old settings
Session + IPC	24	Every phase and failure path, every command registered and its arguments checked
Renderer logic	60	What the card says, when Start is allowed, formatting, change merging
Shared paths	22	POSIX conversion and ignore rules

Not tested here, by design

No real Google account, no real Minecraft server and no installer build were used: those need your accounts and your PC. Do this once with two Google accounts (A owns the server, B is invited) on two PCs or two Windows user accounts:

1 Sign in on both

Allow Drive access. A creates a server, chooses an existing world folder, starts, waits for "Hosting", stops. Check that `.launcher/manifest.json` appears on Drive.

2 Invite and join

A invites B as Writer. B joins with the link, picks an empty folder. B's card should say "First sync on this PC" with the world size.

3 Take turns

B starts (download), plays, places a block, stops. A's card should say "B played after you". A starts and sees the block.

4 The lock

While A hosts, B's Start is disabled with "A is hosting the server". Kill A's app from Task Manager: after about 45 s B can start.

5 Ownership

B deletes a region file that A uploaded, then plays and stops. On Drive the file is in `.launcher/pending-delete` with 0 bytes. A's card says it waits; when A opens the app it is deleted for good.

6 Conflict

Both edit `server.properties` offline. The second to open sees "Both changed"; choose "Keep cloud version" and check the backups folder (Directory menu).

7 Crash during upload

Stop a big world and pull the network cable during the upload. Expect "upload did not finish" with Retry. Close the app, reconnect, open it: the upload finishes on its own and nothing is duplicated on Drive.

10 The commits

First your work was saved and merged, exactly as you chose: the uncommitted changes became a snapshot commit on `rewritten-manifest`, and that branch was merged into main, keeping your hand-written version where the two differed. The rewrite follows in steps that each build on the last. Nothing was pushed; `git log --oneline 33bb9d1..main` shows them.

COMMIT	WHAT IT CONTAINS
e63130c	Your uncommitted work, saved as a snapshot on rewritten-manifest
33bb9d1	Merge of rewritten-manifest into main (your hand-written version kept)
9d4eebd	Foundation: shared types and IPC contract, Drive interface, fake Drive, dependencies, the spec and the research notes
46936d5	Sync core: POSIX paths, three-way planner, MD5 scanner, manifest format 2
52f1e8c	Lock service: lease, renewal, claim-then-verify, Google's clock
2c1d2c1	Drive client, OAuth and the transfer engine
ee4fc3a	Minecraft and playit processes, app data, settings and servers
993ed3c	Sync engine and change tracker
7c5a950	Session controller, typed IPC, the new main.ts and preload; old services removed
458d572	Renderer: stores, sync card, transfers, rules editor; manual transfer buttons removed
9f57ff4	Console scrolling; template builder config removed
17186a4	Reuse of the last scan (no double hashing)
dec885e	Readers cannot host, and are told why
ff2b97d	Tests for the Google sign-in code
97f319e	The Leave dialog explains storage (N22)
last	This report and Parts 1 and 2, in `docs/architecture-review/`

> YOUR OLD DATA
>
> On first start the app moves your old settings (folder paths, playit.gg path, RAM, blacklist) into the new per-server files; the old blacklist becomes the starting shared ignore list. Old `lock.json` and `manifest.json` files at the root of a server folder are ignored and can be deleted by hand.

Part 1: Drive Launcher As-Built (findings F1–F41). Part 2: Drive Launcher Blueprint (design). This document: the changes made on main, 16 October 2026.